

Python Foundations Challenge

DATA TYPES • VARIABLES • MUTABILITY • SETS

Level: Advanced

4 Tasks

Instructions: Test your understanding of standard Python behaviors, memory assignment, set operations, and type mutability. Attempt to answer all questions before reviewing the solutions on Page 2.

Task 1: The Trap of Immutable Mutables

HARD

Analyze the code below. Predict the exact terminal output and state if any exceptions occur. Explain the underlying memory behavior.

```
# Predict the output and behavior of this block data = (1, 2, [3, 4]) data[2].append(5) print("Point A:", data) try: data[2] +
```

Task 2: Deep Set Deduplication

HARD

Write a function `unique_elements(lst)` that returns a set of unique items from an arbitrary list. Handle unhashable types (like lists and dicts) by converting them into immutable hashable representations without crashing.

```
# Sample Input raw_data = [1, "1", (1, 2), [1, 2], {"a": 1}, {"a": 1}, [1, 2], True] # Desired output set containing hashable r
```

- Convert lists to tuples recursively if needed.
- Convert dictionaries to sorted tuples of `(key, value)` pairs.
- Note that in Python: `True == 1` and `hash(True) == hash(1)`.

Task 3: Integer Caching & Identity

MEDIUM

Predict the evaluation (`True` or `False`) of both conditions in standard interactive CPython. Explain the core difference between `==` and `is`.

```
a = 256 b = 256 print(a is b) # Condition 1 x = 257 y = 257 print(x is y) # Condition 2
```

Task 4: Advanced Set Operations Without Loops

MEDIUM

Given three sets representing course enrollments, construct single-line set expressions (no loops, no comprehensions) for the required targets:

```
math = {"Alice", "Bob", "Charlie", "David"} physics = {"Charlie", "David", "Eve", "Frank"} cs = {"David", "Frank", "Grace",
```

- **Target A:** Students enrolled in *exactly two* courses.
- **Target B:** Students enrolled in *at least one* course, but *not all three*.

Solutions & Explanations

COMPREHENSIVE TECHNICAL ANALYSIS

[Answer Key](#)

Task 1 Solution: Augmented Assignment on Tuple Elements

Output:

```
Point A: (1, 2, [3, 4, 5]) Caught TypeError! Point B: (1, 2, [3, 4, 5, 6, 7])
```

Explanation:

- `data[2].append(5)` succeeds because it modifies the internal list in-place without altering the tuple's reference pointer.
- `data[2] += [6, 7]` performs two steps under the hood:
 - It executes `data[2].extend([6, 7])`, which mutates the list in place (succeeds!).
 - It attempts to reassign the result back to `data[2] = list_ref`. Because `data` is a tuple, assignment fails and raises a `TypeError`. Thus, the mutation occurs *before* the exception is thrown.

Task 2 Solution: Deep Set Deduplication

```
def make_hashable(item): if isinstance(item, list): return tuple(make_hashable(i) for i in item) if isinstance(item, dict): re
```

Explanation: Sets require elements to be hashable (immutable). The helper recursively turns lists into tuples and dictionaries into sorted key-value tuple pairs. `True` is treated as identical to `1` because in Python `bool` is a subclass of `int` and `1 == True` evaluated to hash collisions.

Task 3 Solution: Integer Caching Mechanisms

Output: `True` followed by `False` (in interactive REPL).

Explanation:

- `==` checks for **value equality** (do both variables hold equal data?).
- `is` checks for **object identity** (do both variables point to the exact same memory address `id()`?).
- CPython pre-allocates and caches small integers in the range `[-5, 256]`. Thus, `a` and `b` point to the exact same cached object in memory. `257` falls outside this range, creating distinct objects.

Task 4 Solution: One-Line Set Expressions

Target A (Exactly 2 courses):

```
target_a = (math & physics) ^ (physics & cs) ^ (math & cs) ^ (math & physics & cs) # Alternative using set operators: targ
```

Target B (At least 1, but not all 3):

```
target_b = (math | physics | cs) - (math & physics & cs)
```